Lezione 1 – Benvenuti in Java

Il corso, la piattaforma Java, il primo programma

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

25 settembre 2026

Agenda della lezione (2h)

1. **Il corso** (*15 min*)
   - obiettivi, percorso, organizzazione, homework
2. **Java: perché e come** (*25 min*)
   - "scrivi una volta, esegui ovunque": bytecode e JVM
   - JDK e versioni; installazione su Windows, macOS e Linux
3. **Il primo programma** (*30 min*)
   - anatomia di `HelloWorld.java`
   - edit → `javac` → `java`: dal vivo, da terminale
   - gli errori che farete (tutti) la prima settimana
4. **Esercitazione guidata** – programmi interattivi (*40 min*)
   - `System.out.printf`, `Scanner`, metodi `static`
   - `SommaNumeri` commentato insieme, poi `Media` scritto insieme, passo passo
5. **Wrap-up e homework** (*10 min*)

Obiettivi del corso

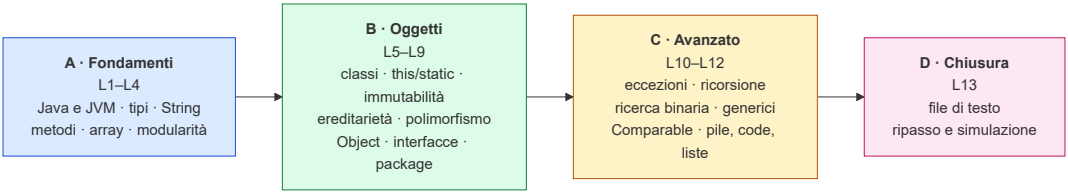
Al termine del corso saprete:

- **progettare** piccoli sistemi software in modo modulare, ragionando per *astrazioni*;
- **programmare a oggetti** in Java: classi, incapsulamento, ereditarietà, polimorfismo, interfacce;
- usare alcune **tecniche avanzate**: ricorsione, eccezioni, tipi generici, API standard;
- implementare le **strutture dati fondamentali**: pila, coda, liste collegate;
- leggere e scrivere **file di testo**.

È un laboratorio

Ogni lezione contiene codice scritto *insieme* e codice scritto *da voi*. Portate il portatile con il JDK installato (oggi vediamo come).

Il percorso: 13 lezioni, 4 blocchi



Martedì, 2h, fino al 15 dicembre (l'8 dicembre è festivo), più alcuni laboratori di sola esercitazione. Il programma completo, con date e contenuti di ogni lezione, sarà pubblicato sul team Microsoft Teams del corso (seguirà comunicazione).

Come funziona una lezione – e il corso

Struttura tipica (2h)

- 10–15' ripasso e correzione homework
- 35–45' concetti nuovi, con esempi *eseguiti*
- 50–60' esercitazione
- 5–10' wrap-up e homework

Strumenti

- JDK 25 (LTS, Eclipse Temurin)
- Visual Studio Code con l'*Extension Pack for Java* (lo installiamo oggi)
- terminale per capire cosa fa l'IDE "sotto"

Homework

- uno a settimana, da svolgere prima della lezione successiva
- non obbligatori ma **fortemente consigliati**
- all'inizio di ogni lezione chiederò a qualcuno di **presentare la propria soluzione**: la discutiamo insieme

Testo di riferimento

- Deitel & Deitel, *Programmare in Java*, 11ª ed., Pearson
- slide, codice ed esercizi sulla repository GitHub del corso: `https://github.com/Uzarel/lab-programmazione-2026-27`

Che cos'è Java

- Nasce in Sun Microsystems (progetto *Green*, 1991; "Oak"), annunciato ufficialmente il 23 maggio 1995. Dal 2006–2007 l'implementazione di riferimento è *open source* (OpenJDK).
- È un linguaggio **orientato agli oggetti**, **fortemente tipizzato**, con **gestione automatica della memoria** (garbage collector).
- È definito da un documento, la *Java Language Specification* (JLS); la piattaforma è **Java SE** (Standard Edition), un insieme di specifiche mantenute dal Java Community Process.

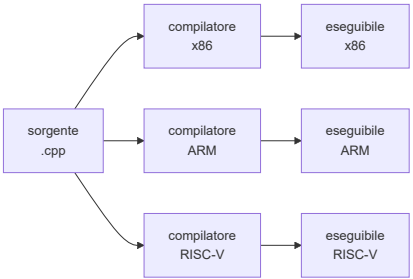
La filosofia: *Write Once, Run Anywhere*

Un programma Java viene scritto e compilato **una volta** e poi eseguito **ovunque** esista una Java Virtual Machine: Windows, macOS, Linux, Android, server, schede embedded...

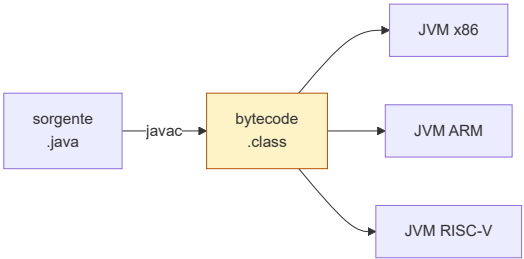
Come è possibile? La risposta è nella parola **bytecode**.

Compilazione tradizionale (es. C/C++) vs Java

C++: un compilatore (e un eseguibile) per ogni architettura



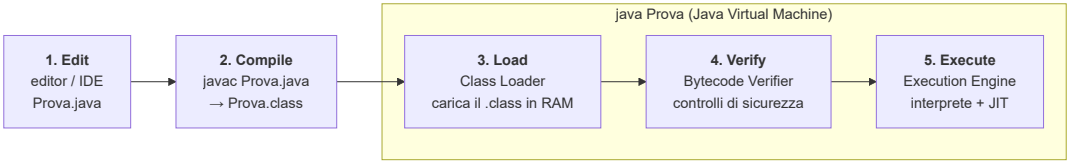
Java: un solo bytecode, una JVM per architettura



Il punto

`javac` **non** genera codice macchina ma *bytecode*, per un processore virtuale. Si compila **una volta**; a tradurlo è la JVM installata su ogni macchina.

Il ciclo di vita di un programma Java: le 5 fasi



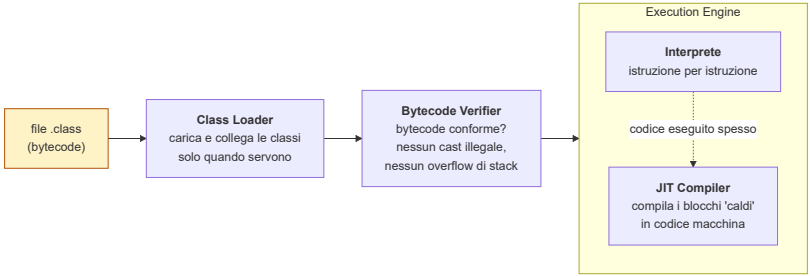
A tempo di compilazione

- controllo sintattico e dei *tipi*
- un file `.class` **per ogni classe**

A tempo di esecuzione

- il *collegamento* è dinamico: le classi sono caricate solo quando servono
- ogni classe caricata viene verificata

Dentro la JVM



- La JVM è **processore virtuale + sistema operativo virtuale**: un ambiente completo e isolato.
- Il *verifier* impedisce al codice di violare l'integrità del sistema (es. trasformare un intero in un puntatore).
- L'*interprete* parte subito; il *JIT* ottimizza le parti eseguite più spesso: oggi Java è competitivo con i linguaggi compilati.
- La **memoria è gestita dalla JVM**: creiamo oggetti con `new`, il *garbage collector* li libera.

JDK e versioni: che cosa installare

JDK – Java Development Kit

- tutto ciò che serve per **sviluppare** ed **eseguire**: il compilatore `javac`, la JVM `java`, gli strumenti (`jshell`, `javadoc`, `jar`...) e la libreria standard
- è l'unica cosa da installare: contiene tutto

La libreria standard

- migliaia di classi organizzate in *package*: `java.lang` (String, Math, System...), `java.util` (Scanner, collezioni), `java.io` (file)...
- documentazione ufficiale: *docs.oracle.com* → Java SE 25 API

Versioni

- una release ogni 6 mesi (marzo e settembre): due numeri di versione all'anno
- **LTS** (Long Term Support) ogni 2 anni, cioè una versione ogni quattro: 17 (2021), 21 (2023), **25** (2025)
- per il corso usiamo tutti **JDK 25**: stessa versione, stessi comportamenti, stessi messaggi d'errore – distribuzione consigliata: Eclipse Temurin (Adoptium)

Installare il JDK 25 (Eclipse Temurin)

Windows – da `https://adoptium.net` scaricare *Temurin 25 (LTS)* come `.msi`; nell'installer scegliere *Custom* e attivare **Set JAVA_HOME** e **Add to PATH**.

macOS – prima il chip: menu Apple → *Informazioni su questo Mac*: *Apple M1–M4* → **aarch64**, *Intel* → **x64**. Poi da `https://adoptium.net` il `.pkg` per il proprio chip (doppio clic e avanti), oppure con Homebrew:

```
$ brew install --cask temurin@25
```

Linux – il modo più semplice, uguale per tutte le distribuzioni, è **SDKMAN** (non richiede root):

```
$ curl -s "https://get.sdkman.io" | bash
$ source ~/.sdkman/bin/sdkman-init.sh
$ sdk install java 25-tem
```

In alternativa il pacchetto della distribuzione, se già alla 25: `sudo apt install openjdk-25-jdk` (Ubuntu/Debian), `sudo dnf install java-25-openjdk-devel` (Fedora).

In ogni caso

Serve il **JDK** completo, versione **25**. Dopo l'installazione aprite un **nuovo** terminale: quello già aperto non vede i programmi appena installati.

Verifichiamo l'installazione (adesso, tutti)

Da terminale (macOS/Linux) o prompt dei comandi / PowerShell (Windows):

```
$ java -version
openjdk version "25.0.2" 2026-01-20 LTS
OpenJDK Runtime Environment Temurin-25.0.2+10 (build 25.0.2+10-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.2+10 (build 25.0.2+10-LTS, mixed mode)

$ javac -version
javac 25.0.2
```

Se `java` funziona ma `javac` no

- avete installato solo la runtime: serve il JDK completo

Se compare una versione diversa da 25

- c'è un altro Java installato che viene trovato per primo: su Windows controllate `JAVA_HOME` e l'ordine nel `PATH`; su macOS `/usr/libexec/java_home -V` elenca i JDK presenti

Se nessuno dei due funziona

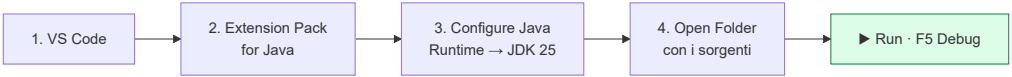
- JDK non installato, oppure non è nel `PATH`
- avete riaperto il terminale dopo l'installazione?

IDE

- useremo sempre **Visual Studio Code**: prossima slide

Visual Studio Code per Java: installazione in 4 passi

1. **Installare VS Code** da `https://code.visualstudio.com`
2. **Installare l'estensione**: icona *Extensions* (`Ctrl+Shift+X`), cercare **Extension Pack for Java** (Microsoft), *Install*. Porta con sé supporto al linguaggio, debugger, test runner e gestione progetti.
3. **Collegare il JDK**: `Ctrl+Shift+P` → *Java: Configure Java Runtime*; deve comparire il JDK 25. Se non lo trova, impostare in *Settings* `java.jdt.ls.java.home` sulla cartella del JDK.
4. **Aprire una cartella** (*File* → *Open Folder*), non un singolo file: VS Code compila da solo ogni `.java` che contiene.



Come si esegue un programma

Sopra il `main` compaiono i link **Run — Debug** (oppure `F5`). L'output appare nel pannello *Terminal*: sotto, VS Code lancia lo stesso `javac` + `java` che oggi vediamo a mano.

HelloWorld.java

```java
/* Prima classe di esempio: illustra il metodo main
   e i passi di sviluppo di un'applicazione Java. */
public class HelloWorld {

    // metodi della classe
    public static void main(String[] args) {
        System.out.println("Welcome to Java!");
    }
}
```

Un'**applicazione** Java è un programma eseguibile autonomamente. Ha bisogno di:

- almeno una **classe** (qui `HelloWorld`) dichiarata dal programmatore...
- ...che contenga un metodo `main` con *esattamente* questa firma: è il punto di avvio.

Anatomia riga per riga

```
public class HelloWorld {
```

- `class` introduce la dichiarazione di classe, seguita dal **nome**
- `public` è un *modificatore di accesso* (ne parleremo)
- le graffe { } delimitano il **corpo** della classe

```
public static void main(String[] args)
```

- `public`: invocabile dall'esterno (dalla JVM)
- `static`: invocabile **senza creare oggetti**
- `void`: non restituisce nulla
- `String[] args`: gli argomenti da riga di comando

```
System.out.println("Welcome to Java!");
```

- `System.out` è l'oggetto *standard output*
- `println` è un suo **metodo**: stampa l'argomento e va a capo (`print` non va a capo)
- `"Welcome to Java!"` è una **stringa letterale**
- ogni *istruzione* termina con `;`

```
/* commento su più righe */
// commento fino a fine riga
```

I commenti sono ignorati dal compilatore. Usateli per spiegare il *perché*, non il *cosa*.

Regole e convenzioni sui nomi

Regole (le impone il compilatore)

- un identificatore è formato da lettere, cifre, `_` e `$`
- non può iniziare con una cifra né contenere spazi
- non può essere una parola chiave (`class`, `public`, `int`...)
- Java è **case sensitive**: `a1` e `A1` sono identificatori diversi
- il file `X.java` deve contenere la classe pubblica `X` (stesso nome, stesse maiuscole)

Convenzioni (le impone la comunità)

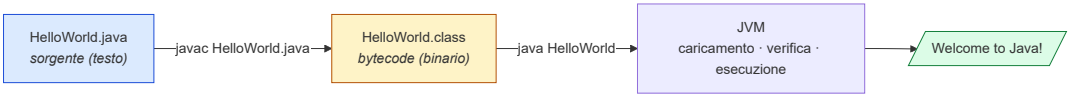
- classi: `UpperCamelCase` → `ContoBancario`
- metodi e variabili: `lowerCamelCase` → `calcolaMedia`, `numeroStudenti`
- costanti: `UPPER_SNAKE_CASE` → `MAX_TENTATIVI`
- package: tutto minuscolo → `java.util`

Perché rispettarle

Il compilatore non se ne accorge, ma chi legge il vostro codice (incluso il voi del futuro) sì. All'esame le convenzioni fanno parte della valutazione.

Dal sorgente all'output: dal vivo, da terminale

```
$ ls
HelloWorld.java
$ javac HelloWorld.java          # compilazione: produce il bytecode
$ ls
HelloWorld.class  HelloWorld.java
$ java HelloWorld                # esecuzione: nome della CLASSE, senza .class
Welcome to Java!
```



Il file `.class` **non** è un file di testo: aprirlo con un editor mostra caratteri senza senso. Provate.

Gli errori della prima settimana (e come leggerli)

1. Nome del file diverso dalla classe

```
$ javac hello.java
hello.java:1: error: class HelloWorld is
public, should be declared in a file
named HelloWorld.java
```

2. Manca un ; o una graffa

```
HelloWorld.java:6: error: ';' expected
    System.out.println("Welcome")
                                  ^
```

Il compilatore indica *file:riga* e mette ^ dove si è accorto del problema (non sempre dove sta l'errore!).

3. Maiuscole sbagliate

```
HelloWorld.java:6: error: package system
does not exist
    system.out.println("Welcome");
```

4. Eseguire con `.class` o senza compilare

```
$ java HelloWorld.class
Error: Could not find or load main class
HelloWorld.class
```

5. Firma del `main` sbagliata

```
Error: Main method not found in class
HelloWorld, please define the main method
as: public static void main(String[] args)
```

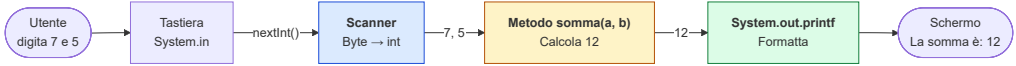
Regola d'oro

Leggete **il primo** messaggio d'errore, non l'ultimo: gli errori successivi sono spesso conseguenza del primo.

Obiettivo dell'esercitazione

Finora il programma *parla* soltanto (`println`). Ora vogliamo programmi che **dialogano** con l'utente:

1. **leggono** numeri e testo digitati da tastiera;
2. li **elaborano** in metodi separati, ognuno con un compito preciso;
3. **stampano** il risultato ben formattato.



Servono tre ingredienti nuovi, che vediamo uno alla volta e poi mettiamo insieme in `SommaNumeri`:

- `System.out.printf` – stampa formattata (colonne, decimali)
- `Scanner` – lettura da tastiera
- metodi `static` – "funzioni" per spezzare il programma in compiti

Ingrediente 1 – Stampa formattata con `printf`

Si comporta come la `printf` del C:

```
int n = 7;
double media = 27.3333;
String nome = "Anna";
System.out.printf("Ciao %s, hai %d esami con media %.2f%n", nome, n, media);
// Ciao Anna, hai 7 esami con media 27.33
```

Specificatore	Significato
`%d`	intero
`%f`, `%.2f`	reale (con 2 decimali)
`%s`	stringa
`%c`	carattere
`%n`	a capo (portabile: usatelo al posto di `\n`)
`%5d`, `%-10s`	larghezza minima 5, allineato a sinistra su 10

In alternativa: `System.out.println("Ciao " + nome + ", media " + media);` – l'operatore + concatena stringhe (ne parleremo la prossima lezione).

Ingrediente 2 – Leggere dall'utente con Scanner

```
import java.util.Scanner;   // 1. la classe Scanner sta nel package java.util

public class Eta {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);   // 2. un oggetto Scanner "collegato"
    alla tastiera
        System.out.print("Come ti chiami? ");
        String nome = in.nextLine();           // 3. legge un'intera riga
        System.out.print("Quanti anni hai? ");
        int eta = in.nextInt();                // 4. legge il prossimo intero
        System.out.printf("Ciao %s, tra un anno avrai %d anni.%n", nome, eta + 1);
        in.close();                            // 5. chiusura (buona pratica)
    }
}
```

- `import`: dice al compilatore dove trovare `Scanner`
- `new`: **crea un oggetto** (lo studieremo a fondo); `System.in` è lo standard input

Metodi principali: `nextInt()`, `nextDouble()`, `next()` (una parola), `nextLine()` (una riga), `hasNextInt()` (c'è un intero da leggere?)

Ingrediente 3 – Metodi `static`: funzioni per organizzare il codice

```java
public class Geometria {

    // modificatori, tipo, nome, parametri
    public static double areaCerchio(double r) {
        return Math.PI * r * r;
    }

    public static void stampaSaluto() {  // void
        System.out.println("Ciao!");
    }

    public static void main(String[] args) {
        stampaSaluto();                  // chiamata
        double a = areaCerchio(2.0);  // 12.566...
        System.out.printf("Area: %.3f%n", a);
    }
}
```

`static` significa che il metodo **appartiene alla classe**, non a un oggetto: si può chiamare senza fare `new`.

Per ora *tutti* i nostri metodi saranno `static`: sono le "funzioni" che conoscete dal C. Dalla lezione 5 scopriremo i metodi *di istanza*.

Perché spezzare il codice in metodi

- un metodo = **un compito**, con un nome che lo descrive
- si *riusa* e si *testa* separatamente
- il `main` diventa la "trama", leggibile in 5 righe

Commentiamo insieme – SommaNumeri

```java
import java.util.Scanner;

public class SommaNumeri {

    public static void main(String[] args) {
        int risultato = sommaDueNumeri();
        System.out.printf("La somma è: %d%n", risultato);
    }

    public static int sommaDueNumeri() {
        Scanner scanner = new Scanner(System.in);
        System.out.printf("Primo numero: ");
        int numero1 = scanner.nextInt();
        System.out.printf("Secondo numero: ");
        int numero2 = scanner.nextInt();
        scanner.close();
        return numero1 + numero2;
    }
}
```

Leggiamolo dall'alto

1. il `main` *chiama* `sommaDueNumeri()` e mette il valore restituito in `risultato`
2. `sommaDueNumeri` è `static` e restituisce un `int`
3. lo `Scanner` traduce i byte digitati in un `int` utilizzabile
4. `return` termina il metodo e "consegna" il valore al chiamante

Poi lo miglioriamo insieme

- separare *input* e *calcolo*: `static int somma(int a, int b)`
- leggere i numeri nel `main` e lasciare ai metodi solo il calcolo
- cosa succede se l'utente scrive `ciao`? (lo risolveremo alla lezione 10)

Variante – Argomenti da riga di comando

Il parametro `args` del `main` contiene ciò che scriviamo dopo il nome della classe:

```
public class Saluta {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Uso: java Saluta <nome> [<nome> ...]");
            return;
        }
        for (int i = 0; i < args.length; i++) {   // args.length: quanti argomenti
            System.out.printf("Ciao, %s!%n", args[i]);
        }
    }
}
```

```
$ java Saluta Anna Luca
Ciao, Anna!
Ciao, Luca!
$ java Saluta
Uso: java Saluta <nome> [<nome> ...]
```

Insieme, passo passo (25 min) – `Media.java`

Il problema. Chiedere quanti voti si vogliono inserire, leggerli uno alla volta e stampare la media con 2 decimali.

Prima di scrivere codice, decidiamo insieme:

1. quali **dati** servono (quanti voti? la somma? di che tipo?);
2. quale **calcolo** isolare in un metodo, e cosa deve ricevere e restituire
   → `static double media(double somma, int n);`
3. cosa fa il `main`: crea lo `Scanner`, legge, chiama il metodo, stampa.

Come procediamo

Lo scriviamo **insieme**, una riga alla volta: io al PC proiettato, voi sul vostro portatile in VS Code. Alla fine compiliamo ed eseguiamo (almeno una volta **da terminale**) e proviamo qualche input strano. Quel che non finiamo in aula, lo finite a casa.

Una possibile soluzione – `Media.java`

```java
import java.util.Scanner;

public class Media {

    // il calcolo, separato dall'input/output
    public static double media(double somma, int n) {
        return somma / n;    // somma e' double: la divisione e' reale
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Quanti voti? ");
        int n = in.nextInt();
        double somma = 0;
        for (int i = 1; i <= n; i++) {
            System.out.print("Voto " + i + ": ");
            somma += in.nextInt();
        }
        System.out.printf("Media: %.2f%n", media(somma, n));
        in.close();
    }
}
```

Perché `somma` è `double` e non `int`? Provate a cambiarlo e a inserire i voti 27 e 28. E cosa succede con $n = 0$?

Riepilogo: cosa abbiamo trattato

Concetto (unità di riferimento)	**Dove compare oggi**
Bytecode, JVM, WORA (U2)	diagrammi C++ vs Java; le 5 fasi; dentro la JVM
JDK vs JRE, versioni LTS (U1, U2)	verifica dell'installazione, `java -version`
Classe, `main`, `System.out` (U1)	anatomia di `HelloWorld`
`javac` / `java`, file `.class` (U1)	compilazione ed esecuzione da terminale
Identificatori, case sensitivity, CamelCase (U1)	regole e convenzioni; errori tipici
`import`, `Scanner`, `printf` (U6)	`Eta`, `SommaNumeri`, `Media`
Metodi `static`, `return` (U6)	`Geometria`, `sommaDueNumeri`, `Media`
`args`, `args.length` (U1)	`Saluta`

Prossima lezione: il sistema dei tipi – primitivi, costanti, riferimenti e `null` – e le stringhe (`String`, `StringBuilder`).

Homework 1 (da svolgere entro la prossima lezione)

1. `Calcolatrice.java`: legge due numeri reali e un operatore (`+ - * /`) e stampa il risultato con 3 decimali. Un metodo statico per ogni operazione; gestite la divisione per zero con un messaggio.
2. `Bmi.java`: legge peso (kg) e altezza (m), calcola l'indice di massa corporea BMI $=$ peso/altezza^2 in un metodo `static double bmi(double peso, double altezza)` e stampa la categoria: *sottopeso* se BMI $<$ 18,5; *normopeso* tra 18,5 e 24,9; *sovrappeso* tra 25 e 29,9; *obesità* se BMI $\geq$ 30.
3. `Fizzbuzz.java`: legge n da `args` (default 30) e stampa i numeri da 1 a n sostituendo i multipli di 3 con `Fizz`, di 5 con `Buzz`, di entrambi con `FizzBuzz`.

Come funziona

Non è obbligatorio, ma fortemente consigliato: alla prossima lezione chiederò a qualcuno di aprire il proprio codice e presentarlo alla classe, e lo discutiamo insieme. Tenete i file `.java` in una cartella `hw1/` del progetto VS Code; ogni file deve compilare ed eseguire. Nomi di classi, metodi e variabili secondo le convenzioni viste oggi.

Domande?

Buon lavoro!

mattia.fonisto@unina.it